

FORMATION EMBARQUÉE

Antisèche — Vhdl

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Antisèche — VHDL

Rappel permanent : le VHDL **décrit un circuit**, il ne s'exécute pas. Question à se poser à chaque ligne : *quel matériel je fabrique là ?*

Squelette

```
library ieee;
use ieee.std_logic_1164.all;    -- std_logic, std_logic_vector
use ieee.numeric_std.all;       -- unsigned, signed, arithmétique

entity mon_bloc is
  generic ( LARGEUR : natural := 8 );
  port (
    clk   : in  std_logic;
    reset : in  std_logic;
    d      : in  std_logic_vector(LARGEUR-1 downto 0);
    q      : out std_logic_vector(LARGEUR-1 downto 0)
  );
end entity;

architecture rtl of mon_bloc is
  signal interne : unsigned(LARGEUR-1 downto 0) := (others => '0');
begin
  -- ici les process et affectations concurrentes
end architecture;
```

Types et conversions

```
signal b   : std_logic;           -- '0' '1' 'Z' 'U' 'X' '-'
signal v   : std_logic_vector(7 downto 0); -- bus "neutre" (interfaces)
signal u   : unsigned(7 downto 0);  -- sait compter
signal s   : signed(11 downto 0);   -- complément à 2
signal n   : integer range 0 to 99; -- compteur borné

v <= "10100101";    v <= x"A5";      -- littéraux
u <= u + 1;          -- OK (pas sur slv !)
v <= std_logic_vector(u);           -- conversions EXPLICITES
u <= unsigned(v);
n <= to_integer(u);  u <= to_unsigned(n, 8);
v <= (others => '0'); -- tout à zéro (quelle que soit la largeur)
```

Les 2 process canoniques

```
-- SÉQUENTIEL (registres) : la forme la plus fréquente
process (clk)
begin
    if rising_edge(clk) then
        if reset = '1' then q <= (others => '0');
        elsif en = '1' then q <= d;
        end if;
    end if;
end process;

-- COMBINATOIRE : liste de sensibilité COMPLÈTE + valeur par défaut
process (a, b, sel)
begin
    y <= '0'; -- défaut : évite le latch !
    if sel = '1' then y <= a; else y <= b; end if;
end process;
```

Concurrent (hors process)

```
y <= a and b; -- porte
y <= a when en = '1' else '0'; -- mux conditionnel
with sel select
    y <= a when "00", b when "01", c when others; -- others OBLIGATOIRE
```

Les 5 règles qui évitent 90 % des bugs

1. Un signal n'est piloté que **dans un seul** process (sinon *multiple drivers*).
2. Process combinatoire : sortie affectée dans **tous** les chemins → sinon **latch**.
3. Toute entrée **asynchrone** passe par **2 bascules** (métastabilité).
4. **signal** (<=) se met à jour en fin de delta ; **variable** (:=) immédiatement.
5. Une seule horloge, front montant, partout (conception synchrone).

Motifs réutilisables

```
-- Détecteur de front (1 cycle)
process (clk) begin
    if rising_edge(clk) then prec <= sig; end if;
end process;
front <= sig and not prec;

-- Synchroniseur (entrée externe)
process (clk) begin
    if rising_edge(clk) then s0 <= ext; s1 <= s0; end if;
end process;

-- Diviseur / tick périodique
if cpt = F_CLK/2 - 1 then cpt <= (others=>'0'); tick <= '1';
else cpt <= cpt + 1; tick <= '0'; end if;

-- FSM
type t_etat is (REPOS, TRAVAIL);
signal etat : t_etat := REPOS;
process (clk) begin
    if rising_edge(clk) then
        case etat is
            when REPOS => if go then etat <= TRAVAIL; end if;
            when TRAVAIL => if fini then etat <= REPOS; end if;
        end case;
    end if;
end process;
sortie <= '1' when etat = TRAVAIL else '0'; -- Moore
```

Instanciation

```
u_bloc : entity work.mon_bloc
    generic map ( LARGEUR => 16 )
    port map ( clk => clk, reset => rst, d => data_in, q => data_out );
```

Testbench (obligatoire avant synthèse)

```
entity tb is end entity;                                -- pas de ports
architecture sim of tb is
    signal clk : std_logic := '0';
    signal fini : boolean := false;
begin
    dut : entity work.mon_bloc port map (...);
    clk <= not clk after 5 ns when not fini else '0';    -- 100 MHz

    stim : process
    begin
        wait for 20 ns;
        assert q = x"00" report "message d'erreur" severity error;
        report "TESTS OK"; fini <= true; wait;
    end process;
end architecture;
```

Sévérités

note **warning** **error** **failure** (stoppe la simu)

Synthétisable ou non

✓ Synthétisable

✗ Simulation seulement

if case for..generate

wait for 10 ns

rising_edge(clk)

after 5 ns dans une affectation

opérateurs logiques/arith.

report , assert (ignorés)

division par 2ⁿ (décalage)

division/modulo quelconque

GHDL en 4 commandes

```
ghdl -a --std=08 design.vhd tb.vhd    # analyser (sources AVANT testbench)
ghdl -e --std=08 tb                    # élaborer
ghdl -r --std=08 tb --wave=o.ghw      # simuler
gtkwave o.ghw                         # voir les chronogrammes
```

En ligne, sans rien installer : edaplayground.com (GHDL + EPWave).